

- [Guía: tareas que debes hacer tú \(seguridad y login\)](#)
 - [Tarea 1: Variables de entorno en Vercel](#)
 - [Tarea 2: Tabla de usuarios y primer usuario en Turso](#)
 - [Opción A: Script automático \(recomendado\)](#)
 - [Opción B: Manual \(SQL en Turso\)](#)
 - [Tarea 3: Probar en local \(frontend + API\)](#)
 - [Resumen rápido](#)

Guía: tareas que debes hacer tú (seguridad y login)

Sigue estos pasos en orden. Al final tendrás la API funcionando con login y sin exponer credenciales.

Tarea 1: Variables de entorno en Vercel

1. Entra en vercel.com → tu proyecto **Catalogo**.
2. Ve a **Settings** → **Environment Variables**.
3. Añade estas variables (usa **Production**, **Preview** y **Development** si quieres que funcione en todos los entornos):

Nombre	Valor	Notas
<code>TURSO_DATABASE_URL</code>	<code>https://tu- instancia.turso.io</code>	La URL de tu base Turso (la que ya usabas en la app de escritorio).
<code>TURSO_AUTH_TOKEN</code>	Token que te da Turso	Token de lectura/escritura. Lo obtienes en el dashboard de Turso o con <code>db tokens create</code> .
<code>JWT_SECRET</code>	Una clave larga y aleatoria	Ejemplo: genera una con <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code> y copia el resultado.

4. Guarda los cambios. En el próximo despliegue, la API usará estas variables.

Importante: No uses el prefijo `VITE_` en ninguna de estas variables (solo para el backend).

Tarea 2: Tabla de usuarios y primer usuario en Turso

Tienes dos opciones.

Opción A: Script automático (recomendado)

En la raíz del proyecto, con **Node 18 o superior** y un archivo `.env.local` que contenga al menos:

```
TURSO_DATABASE_URL=https://tu-instancia.turso.io
TURSO_AUTH_TOKEN=tu_token_turso
```

Crea `.env.local` copiando `.env.example` y rellenando esos dos valores (y `JWT_SECRET` si quieres probar la API en local).

Luego, desde la **raíz del proyecto**, ejecuta:

```
node scripts/setup-core-users.js crear
```

(O bien: `npm run setup-users crear`.)

El script lee `.env.local` y crea la tabla `core_users` si no existe. Te pedirá **usuario**, **contraseña** y **email** (opcional) y creará el primer usuario como admin (`is_staff=1`, `is_admin=1`).

Para no escribir a mano, puedes pasar los datos por parámetros:

```
node scripts/setup-core-users.js crear admin miPassword admin@ejemplo.com
```

Para añadir más usuarios más adelante (sin permisos de admin):

```
node scripts/setup-core-users.js añadir
```

Opción B: Manual (SQL en Turso)

1. En el [dashboard de Turso](#) abre tu base de datos y entra en la pestaña **SQL** (o usa `turso db shell nombre-db`).
2. Ejecuta este SQL para crear la tabla:

```
CREATE TABLE IF NOT EXISTS core_users (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  username TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  email TEXT,  
  is_staff INTEGER DEFAULT 0,  
  is_admin INTEGER DEFAULT 0  
);
```

3. Genera el hash de la contraseña en tu máquina (en la raíz del proyecto):

```
node -e "const bcrypt = require('bcryptjs'); console.log(bcrypt.hashSync('TU_CONTRASEÑA_AQUI', 10));"
```

4. Copia el hash y ejecuta en Turso (sustituye `<HASH>` y el email si quieres):

```
INSERT INTO core_users (username, password_hash, email, is_staff, is_admin)  
VALUES ('admin', '<HASH>', 'admin@ejemplo.com', 1, 1);
```

Tarea 3: Probar en local (frontend + API)

1. Asegúrate de tener `.env.local` en la raíz con al menos:

- `TURSO_DATABASE_URL`
- `TURSO_AUTH_TOKEN`
- `JWT_SECRET`

2. (Opcional) Instala Vercel CLI globalmente: `npm i -g vercel`

3. En la raíz del proyecto ejecuta:

```
npx vercel dev
```

(O bien: `npm run dev:full`.)

4. Abre la URL que te indique (suele ser `http://localhost:3000`). Verás el frontend y las rutas `/api/*` funcionando.

5. Prueba el login: en tablet/escritorio debería aparecer "Iniciar sesión"; usa el usuario y contraseña que creaste en la Tarea 2.

Resumen rápido

Tarea	Qué hacer
1. Vercel	Añadir <code>TURSO_DATABASE_URL</code> , <code>TURSO_AUTH_TOKEN</code> , <code>JWT_SECRET</code> en Settings → Environment Variables.
2. Turso	Crear tabla <code>core_users</code> y al menos un usuario (script <code>scripts/setup-core-users.js</code> o SQL manual).
3. Local	Crear <code>.env.local</code> con las mismas variables y ejecutar <code>npx vercel dev</code> para probar.

Si algo falla, revisa `docs/API-AUTH-SETUP.md` y que las variables no tengan espacios ni comillas de más en Vercel.